

Verilog Syntax Guide for 8254 PIT Lab

Microprocessors and Peripherals Laboratory

Overview

This guide provides a quick reference to the essential Verilog HDL syntax patterns. It covers register assignments, control flow, bits manipulation, and module instantiations using generic examples.

1 Registers and Wires

In Verilog, signals are typically declared as either `wire` or `reg`:

- **wire:** Used for continuous assignments and connecting modules together. Cannot store a value.
- **reg:** Used inside procedural blocks (e.g., `always` blocks) to store a value across clock cycles.

```
// Declaring a 16-bit register
reg [15:0] timer_val;

// Declaring an 8-bit wire for an internal bus
wire [7:0] data_bus;

// Single bit register flag
reg is_active;
```

2 Assignments: Blocking vs Non-Blocking

Inside an `always` block triggered by a clock edge, you should almost always use **Non-Blocking** assignments (`<=`). This ensures all registers update simultaneously, mimicking physical hardware flip-flops.

```
always @(posedge clk) begin
    // Non-blocking (<=) : Updates concurrently at the clock edge
    timer_val <= timer_val - 1;
    is_active <= 1'b0;
end
```

3 If-Else Statements

Use logical operators (`==`, `!=`, `&&`, `||`, `<=`, `>=`) when making decisions inside an `always` block. Remember to use `begin` and `end` if there is more than one statement in a branch

```
if (timer_val <= 16'd5) begin
    // Execute these lines if condition matches
    is_active <= 1'b0;
    timer_val <= 16'hFFFF;
end else if (timer_val > 16'd5) begin
    timer_val <= timer_val - 1;
    is_active <= 1'b1;
end else begin
    // Default fallback
    is_active <= 1'b1;
end
```

4 Case Statements

```
case (selector_bits[1:0])
    2'b00: register_A <= wire1; // Route to A
    2'b01: register_B <= wire1; // Route to B
    2'b10: register_C <= wire1; // Route to C
    default: ; // Do nothing for unmapped states
endcase
```

5 Bitwise Concatenation {}

To join multiple smaller vectors into a larger one, use curly braces {}.

```
// Combine two registers into a bigger assignment, Assuming
// 'high_byte' and 'low_byte' are 2-bits wide, timer_val is 4-bit
timer_val <= {high_byte, low_byte};

// Example: using only the amount of bits you want
timer_val <= {high_byte, timer_val[3:0]};
```

6 Module Instantiation (Wiring)

When building a top-level block, you need to instantiate your submodules and wire their ports to the appropriate signals.

It is recommended to use Named Port Connections (`.port_name(signal_name)`) rather than relying on order:

```
// Instantiate a 'sensor_controller' module and name the instance
// 'u1'
sensor_controller u1 (
    .clk(sys_clk), // Connect external 'sys_clk' to port 'clk'
    .enable(sensor_en), // Connect 'sensor_en' to port 'enable'
    .data_in(internal_bus), // Connect internal wire to data port
    .mode_select(mode_cfg), // Connect config register output
    .alert(alert_flag) // Connect submodule output to alert_flag
);
```